Projet tutoré analyse initiale

S9 : Création d'une application en VR

I. Description détaillée du sujet

Ce projet a pour but de réaliser une application en VR qui permet de se déplacer dans la quatrième dimension (4D) et de visualiser des formes géométriques en 4D.

Pour ce faire, nous allons utiliser un casque VR (Meta Quest 3) pour permettre la visualisation des objets géométriques. Pour le développement de l'application en elle-même, nous utiliserons le langage JavaScript, et plus précisément la bibliothèque Babylon.js. Cette bibliothèque est un moteur de rendu 3D pour les navigateurs web qui peut être utilisé pour du rendu 3D dans des casques VR. Cependant, il faudra gérer nous même le fait de tourner autour des figures. (changement de point de vue) car Balylon.js ne prend pas en compte la gestion de forme en 4D.

Dans un premier temps, nous allons devoir apprendre à utiliser la bibliothèque Babylon.js et essayer de faire des programmes simples pour nous initier (par exemple un affichage d'une forme en 3D, de pouvoir déplacer une caméra dans un espace 3D etc…).
Une fois que nous maîtriserons l'utilisation de de Babylon.js, nous allons devoir exporter nos programme JS dans le casque VR pour voir comment se comporte le casque VR en y important un programme JS.
Pour finir, nous devrons essayer d'imaginer une représentation de la quatrième dimension dans l'espace 3D. Nous devrons coder la façon dont une forme en 4D doit être déplacée, tournée et visualisée.

Ce sujet comprend une grosse partie technique et mathématiques de part la gestion et la visualisation d'un objet 4D dans un environnement 3D. La partie mathématique portera principalement sur l'utilisation de matrice pour effectuer une rotation d'un objet en 4D.

II. Objectifs

Le projet commence par apprendre à utiliser Babylon.js et développer l'aspect mathématique en amont lors de l'analyse initiale.

Par la suite, nos objectifs seront :

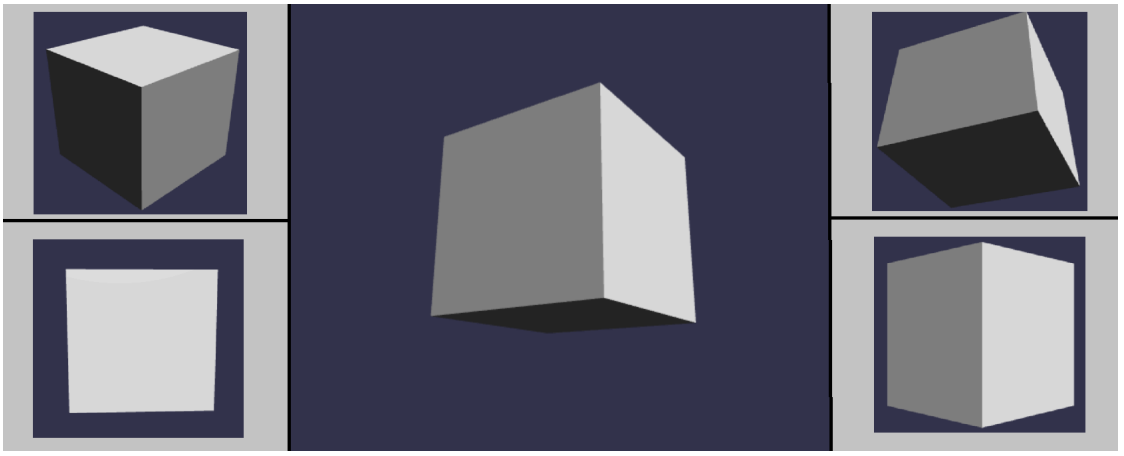
- Modéliser une scène Babylon.js en 3D
- Rotation d'un objet 3D à l'aide de matrices
- Translation d'un objet 3D
- Projection d'une forme 4D (en 3D) dans la scène Babylon (1*)
- Rotation/Translation de la forme 4D
- Affichage des faces visibles uniquement
- Affichage d'une forme 3D en VR
- Gestion des déplacements en VR (manettes/caméra)
- Affichage d'une forme 4D en VR
- Affichage d'une interface des différents plans de la figure 4D (2*)

(1*) Pour afficher une forme en 4D :

- Définition de ses points (reliés par des arêtes)
- Calcul de la distance par rapport à l'objectif (caméra)
- Projection en 3d de la forme
- Affichage de la nouvelle forme projetée

(*) Nous avons proposé cette fonctionnalité au tuteur l'ayant validée :

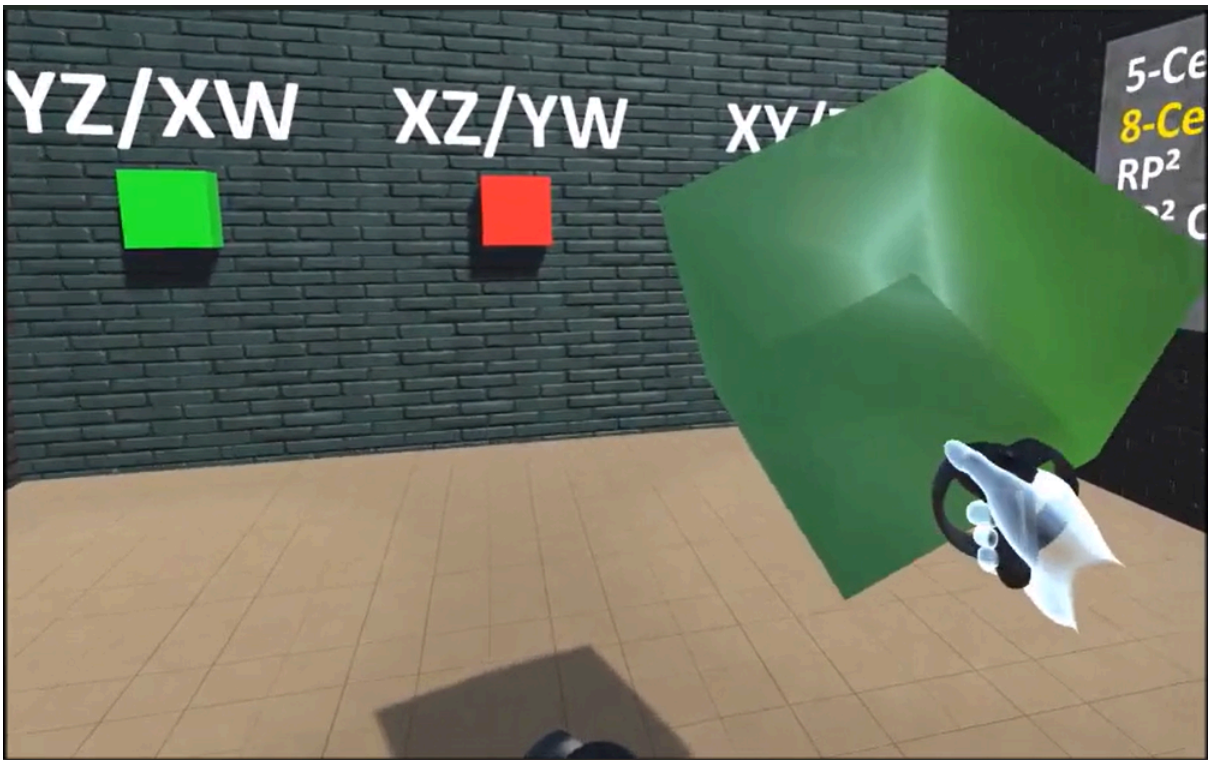
A l'aide d'une manette en VR, pointer la figure géométrique permet d'afficher à l'écran les différents plans de l'objet tel quel :



Lorsque l'utilisateur se déplacera autour de la figure géométrique au milieu et en tournant la figure, tous les autres points de vue à gauche et à droite changeront aussi.

III. Étude de l'existant

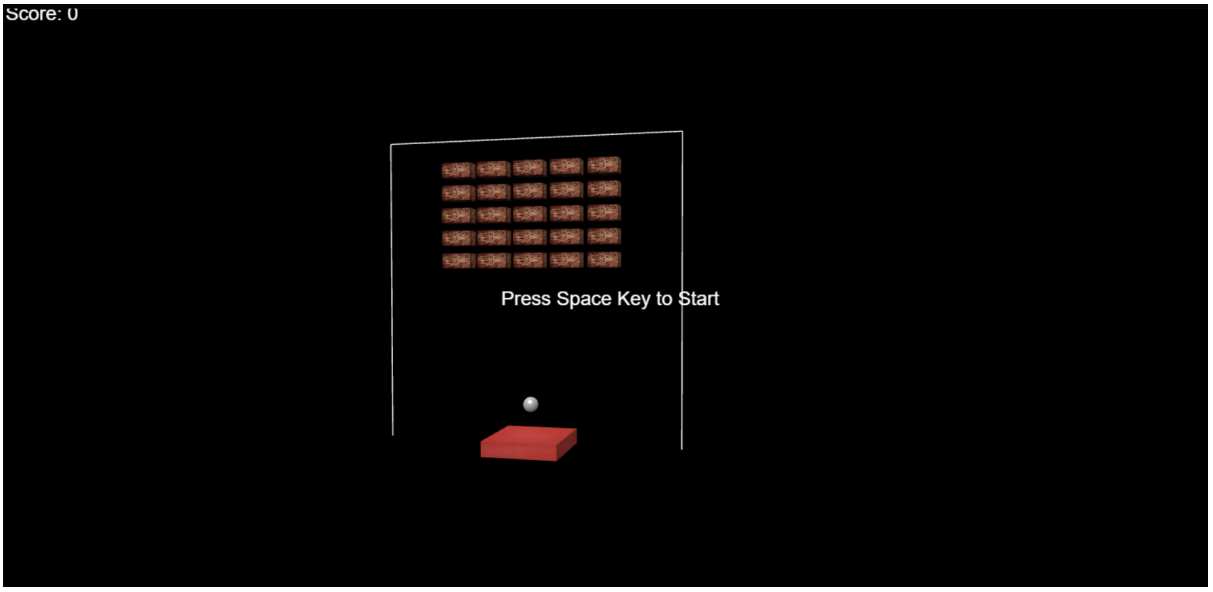
https://youtu.be/1Zamk9ZOrjk?si=dRbxmKKnmsOTaYjZ
Brandon Reichman (VR 4D sous Unity)

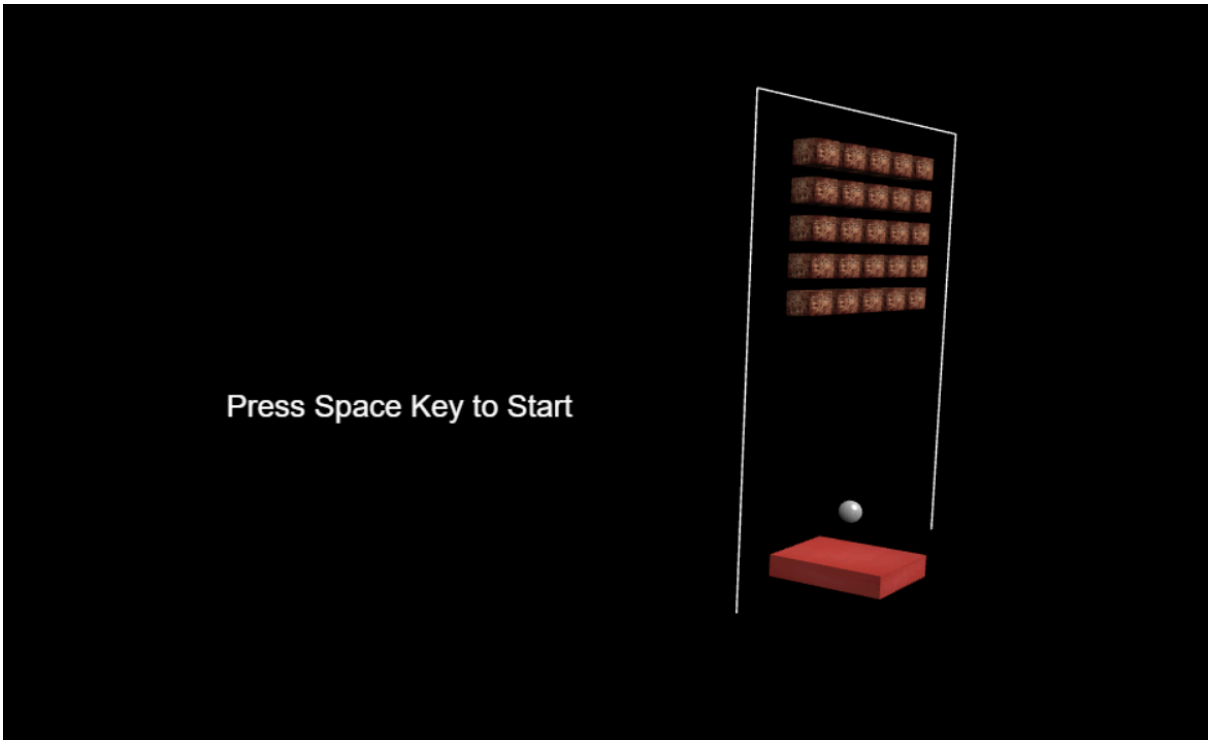


Cette vidéo sous Unity est un bon point de départ, avec la possibilité de choisir une forme géométrique et de la faire tourner autour d'un plan.
Reproduire une telle interface pourrait être notre premier gros objectif.

En fonction du temps pris pour apprendre à manipuler Babylon.js et l'appliquer en réalité virtuelle, nous pourrons nous intéresser à des problèmes plus complexes.

https://github.com/bitlesstec/babylonjs_bricks/tree/master
Exemple d'application 3D avec Babylon.js





Voici un exemple que nous avons trouvé : un casse-brique reproduit en babylon.js
En plus de fonctionner comme un casse-brique classique, il est possible de se déplacer avec les flèches directionnelles et la souris en 3D.

Nous nous inspirerons du code source de ce projet pour apprendre à utiliser Babylon.js durant la première semaine de projet.

Utilisation de [Babylon.js](#) :

documentation :
https://doc.babylonjs.com/features/introductionToFeatures/chap1/first_app

Javascript/HTML + node.js (format web serveur)

Exemples de code utilisant babylon :
//TODO développer l'utilisation de la scène et des objets

```
const createScene =  () => {
    const scene = new BABYLON.Scene(engine);

    const camera = new BABYLON.ArcRotateCamera("camera", -Math.PI / 2, Math.PI / 2.5, 3, new BABYLON.Vector3(0, 0, 0));
    camera.attachControl(canvas, true);

    const light = new BABYLON.HemisphericLight("light", new BABYLON.Vector3(0, 1, 0));

    const box = BABYLON.MeshBuilder.CreateBox("box", {});

    return scene;
}
export default createScene
```

javascript

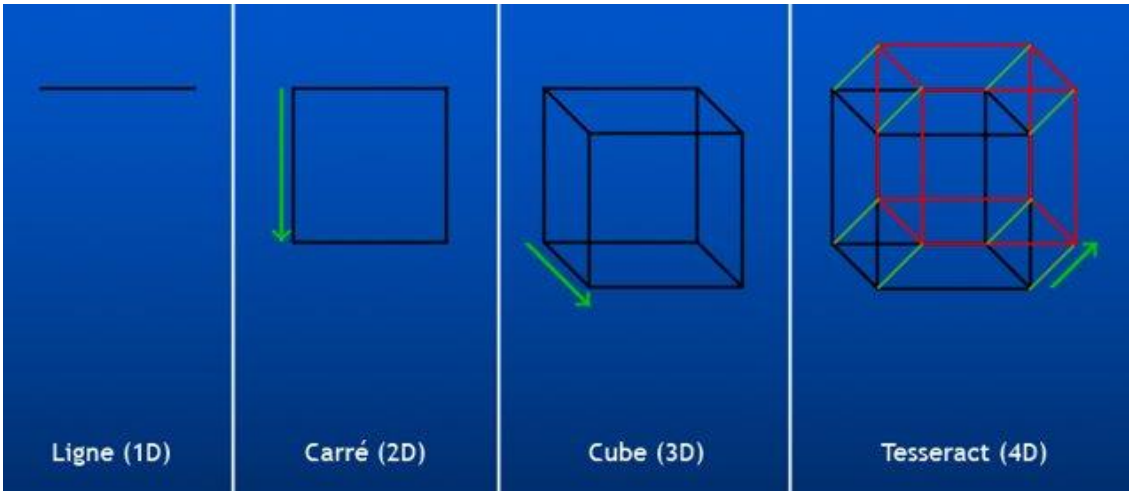
```
var createScene = function () {
  var scene = new BABYLON.Scene(engine);

  // Add a camera to the scene and attach it to the canvas
  // Add a lights to the scene

  //Your Code

  return scene;
};
```

```
import {
    Color4, Engine, Scene, Vector3, FreeCamera,
    HemisphericLight, MeshBuilder, StandardMaterial, Color3, KeyboardEventTypes, Mesh,
    Texture,
    Sound
} from "@babylonjs/core";
// import {  } from "babylonjs";
import * as GUI from '@babylonjs/gui';


const canvas = document.querySelector("#canvas") as HTMLCanvasElement;

const engine = new Engine(canvas, true)

const scene = new Scene(engine);
scene.clearColor = new Color4(0, 0, 0, 1);

const camera = new FreeCamera("camera", new Vector3(0, -1, -5), scene);
camera.attachControl();


const light = new HemisphericLight("light", new Vector3(0, 6, -5), scene);
light.intensity = 1;
```

```
//ball creation
const ball = MeshBuilder.CreateSphere("ball", { diameter: .2 }, scene);

const ballMaterial = new StandardMaterial(`ballMaterial`, scene)
ballMaterial.diffuseTexture = new Texture('./textures/ball.webp', scene)

ball.material = ballMaterial;

let ballMovement = { "h": 0, "v": 0 };

let paddleMovement: number = 0;
const PADDLE_SPD: number = .04;
```

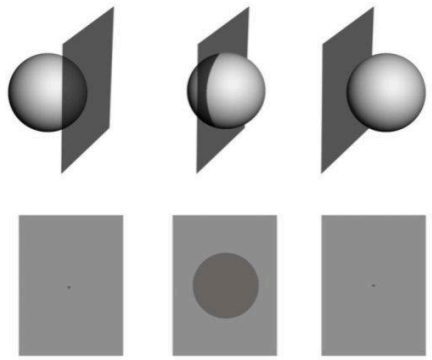
IV. Étude technique sur l'existence de solutions

Passage de la 3D à la 4D :



En ajoutant une dimension à un objet, on copie chaque sommet et le relie à son sommet d'origine et aux sommets adjacents. Le passage à la 4D, 5D et autres dimensions supérieures revient à multiplier le nombre de sommets à chaque itération. Ainsi, le tesseract ou hypercube (cube en 4D) a 16 sommets, comparés aux 8 sommets du cube.

Pour comprendre comment interpréter la 4ème dimension, on peut imaginer l'intersection entre un objet 3D et un plan (2D).



Une sphère devient uniquement un cercle jusqu'à se réduire en un point à travers le plan.

Pareillement, un objet 4D est dans une dimension supérieure à la nôtre et nous ne pouvons que visualiser la projection de cet objet dans notre espace 3D.

Création d'un plan 4D :

Création d'un plan en 4D :

Définition d'un plan : Les vecteurs du plan sont tous orthogonaux
=> produit scalaire entre ces vecteurs est égal à 0)

On définit donc un vecteur comme une matrice 4-1 de 4 vecteurs orthogonaux

u : $\begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$ v: $\begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$ w: $\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$ x: $\begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$

avec

$$\begin{cases} u \cdot v = 0 \\ u \cdot w = 0 \\ u \cdot x = 0 \\ v \cdot w = 0 \\ v \cdot x = 0 \\ w \cdot x = 0 \end{cases}$$

Transformations d'un objet 4D :

On homogénéise les coordonnées d'un point en 4D :

$$P = \begin{bmatrix} x \\ y \\ z \\ w \\ 1 \end{bmatrix}$$

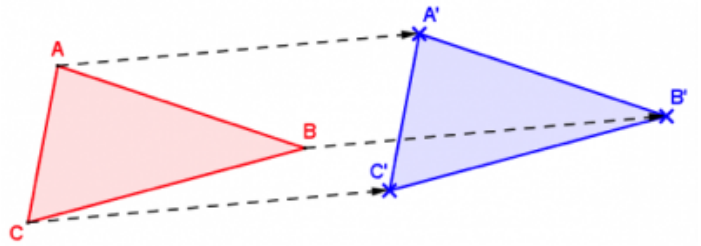
On passe d'une forme P(x,y,z,w) à P(x,y,z,w,1)

Ainsi nous sommes dans un espace projectif et nous pouvons définir chaque transformation comme une opération linéaire sur notre matrice (point P) :

$$p' = T(p)$$

avec p' le transformé de p par la matrice de transformation T.

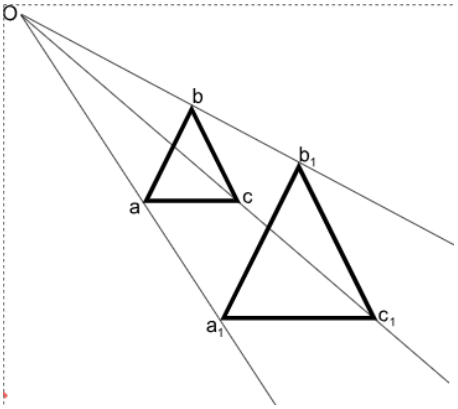
Translation d'un objet 4D :



Une translation revient à faire "glisser" un point, vecteur ou figure.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & t_x \\ 0 & 1 & 0 & 0 & t_y \\ 0 & 0 & 1 & 0 & t_z \\ 0 & 0 & 0 & 1 & t_w \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ w \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ z + t_z \\ w + t_w \\ 1 \end{bmatrix}$$

Homothétie d'un objet 4D :



Une homothétie revient à agrandir ou rétrécir un objet en rapprochant ou éloignant les points de l'objectif O.

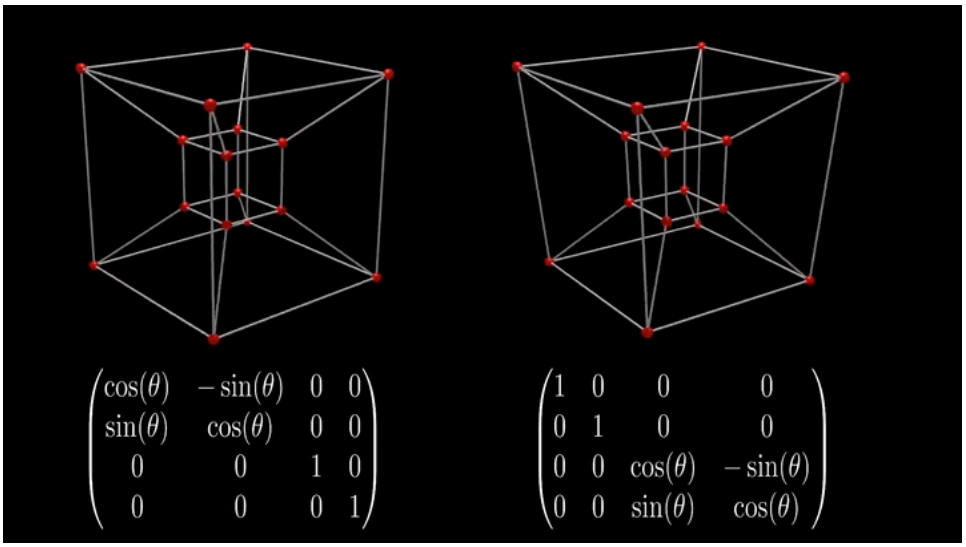
$$H = \begin{bmatrix} k & 0 & 0 & 0 & 0 \\ 0 & k & 0 & 0 & 0 \\ 0 & 0 & k & 0 & 0 \\ 0 & 0 & 0 & k & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$p' = H \cdot \begin{bmatrix} x \\ y \\ z \\ w \\ 1 \end{bmatrix} = \begin{bmatrix} kx \\ ky \\ kz \\ kw \\ 1 \end{bmatrix}$$

Rotation d'un objet 4D :

Simple rotation :

Exemple de rotation d'un hypercube respectivement autour du plan (x, y) et (z, w) :



Pour effectuer une rotation d'objets affichés en 4D, on peut utiliser une matrice de rotation :

exemple de matrices de rotation autour de différents plans (z,w) et (y,z)

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos(\theta) & -\sin(\theta) \\ 0 & 0 & \sin(\theta) & \cos(\theta) \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

on applique à nos 4 coordonnées la matrice de rotation :

$$\begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos(\theta) & -\sin(\theta) \\ 0 & 0 & \sin(\theta) & \cos(\theta) \end{pmatrix}$$

On obtient ainsi nos nouvelles coordonnées.

Le fait de se déplacer autour de formes en 4d revient à changer de perspective. Ce changement s'apparente comme une rotation autour d'un point et on est donc capable de définir cette transformation, et donc d'afficher ce à quoi pourrait ressembler une vision d'une 4ème dimension.

Double rotation :
Pour faire une double rotation, il faut tourner autour de 2 plans orthogonaux (dont le produit scalaire de n'importe quel vecteur des deux plans est nul). Il faut composer les deux matrices de rotation en une seule matrice diagonale :

$$R(\theta, \phi) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & \cos\phi & -\sin\phi \\ 0 & 0 & \sin\phi & \cos\phi \end{pmatrix}$$

rotation selon les plans (x, y) et (z, w)

Projection en perspective 4D -> 3D :
Pour visualiser un objet 4D dans un espace 3D, nous allons utiliser ce type de projection qui permet de mieux visualiser la profondeur 4D (coordonnée w).

Un point en 4D est :

$$P = (x, y, z, w)$$

Pour le visualiser en 3D, il nous faut un point de la forme :

$$P' = (x', y', z')$$

Pour retirer une coordonnée et n'en avoir plus que 3, on peut partir du principe que l'observateur se trouve à une distance d du point P.
On peut donc afficher ce point en divisant la coordonnée par la distance moins la coordonnée w :

$$x' = \frac{x}{d-w}, \quad y' = \frac{y}{d-w}, \quad z' = \frac{z}{d-w}$$

avec d :

$$d = \sqrt{(w_1 - w_0)^2 + (x_1 - x_0)^2 + (y_1 - y_0)^2 + (z_1 - z_0)^2}$$

On peut voir cette opération comme une multiplication par une matrice de projection, puis diviser les 3 premières composantes par la quatrième :

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 & d \end{pmatrix}$$

$$M \cdot \begin{pmatrix} x \\ y \\ z \\ w \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ -w + d \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ d - w \end{pmatrix}.$$